Project

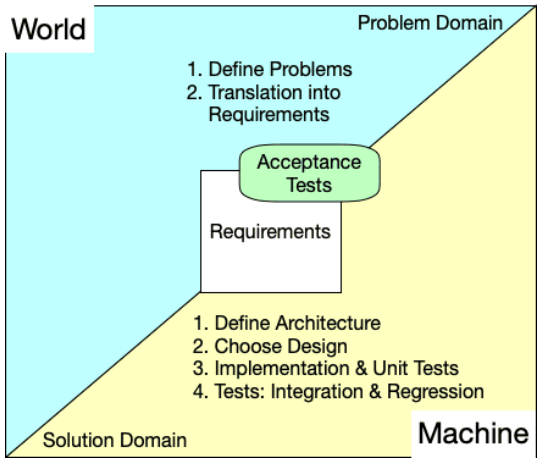
Solving Problems **Individually or as a Team**

From Problem to Software Solution

1. **World → Requirements** — understand the problem; write testable requirements
2. **Requirements → Design** — define architecture (high level) and modules/interfaces (low level)
3. **Design → Implementation** — build, unit-test, and document
4. **Implementation → Validation** — prove the requirements are met

Prototype and validation results may send us back to requirements and design.

The Big Picture

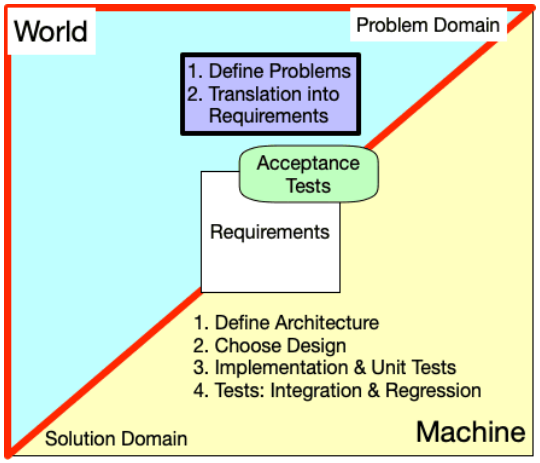


We use `TodoApp` as a small example. The same steps apply to larger projects.

Step 1: World → Requirements

World = the problem and its environment before software exists.

Understand the problem and why it matters.



Understand Why (Why do we need this software?)

Students manage deadlines across courses. They often:

- Forget upcoming deadlines
- Start assignments too late
- Lose track of multiple schedules

They need help managing time and meeting deadlines.

Define Problems: What exactly needs solving?

P1: No Proactive Notifications

Students lack advance deadline alerts.

P2: Limited Accessibility

Students work mainly in browsers but lack a web tool for schedules across devices.

Translate the Problems into Features/Requirements

Turn the **World** (problem domain) into a feature list.

Features

Features describe how the app addresses each problem:

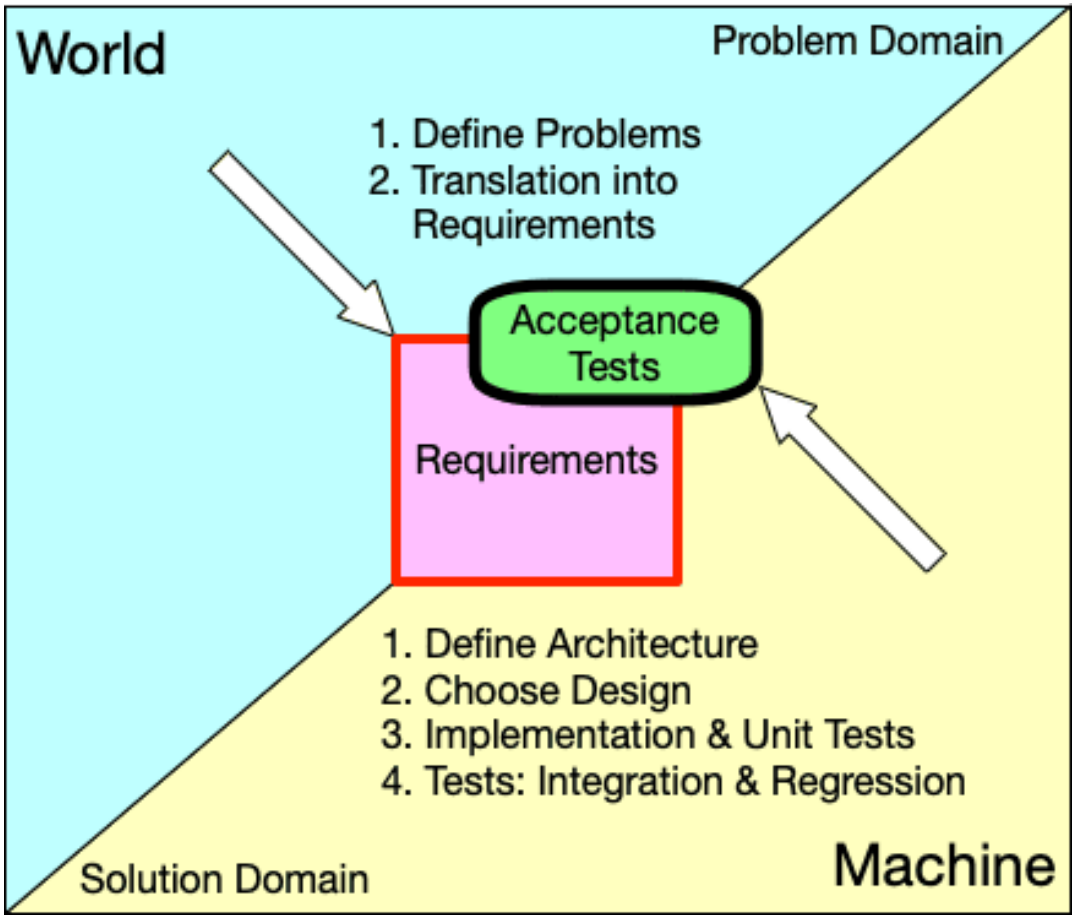
1. Notify students of a deadline N days in advance, by email.
2. Create, read, update, and delete (CRUD) schedules on the web.

Requirements (User Story Mapping)

Break each feature into **user stories**:

1. **Who** the user is
2. **What** they need to do
3. **Why** they need it

Give each story **acceptance criteria**: conditions checked by acceptance tests.


World
Problem Domain
1. Define Problems
2. Translation into Requirements
Acceptance Tests
Requirements
1. Define Architecture
2. Choose Design
3. Implementation & Unit Tests
4. Tests: Integration & Regression
Solution Domain
Machine

Feature 1: Notify students N days before a deadline

S = one schedule/task (e.g., "HW3 due Friday").

- **F1-R1**: **As** a student, I **want** an email N days before schedule S is due, **so that** I don't miss it.
- **F1-R2**: **As** a student, I **want to** mark schedule S as complete, **so that** I stop getting reminders and can track my progress.

F1-R1 is an **epic**: too large for one story. Split it:

- **F1-R1a**: **As** a student, I **want to** set my notification email and N (days before), **so that** the system knows where and when to notify me.
- **F1-R1b**: **As** a student, I **want** the system to automatically send the email N days before the due date, **so that** I'm alerted proactively.

Feature 1 → **3 testable requirements**: F1-R1a, F1-R1b, F1-R2.

- Example criterion for F1-R1b: For an incomplete task due in N days, send one email. Record delivery to prevent duplicate reminders.

Feature 2: CRUD schedules

- **F2-R1**: **As** a student, I **want to** create a schedule, **so that** it's saved online.
- **F2-R2**: **As** a student, I **want to** view saved schedules, **so that** I can track upcoming tasks.

- **F2-R3**: **As** a student, I **want to** update a schedule, **so that** my plan stays accurate.
- **F2-R4**: **As** a student, I **want to** delete a schedule, **so that** I can remove tasks I no longer need.

Two features, seven requirements — each with its own ID:

- Feature 1 → F1-R1a, F1-R1b, F1-R2 *(3)*
- Feature 2 → F2-R1, F2-R2, F2-R3, F2-R4 *(4)*

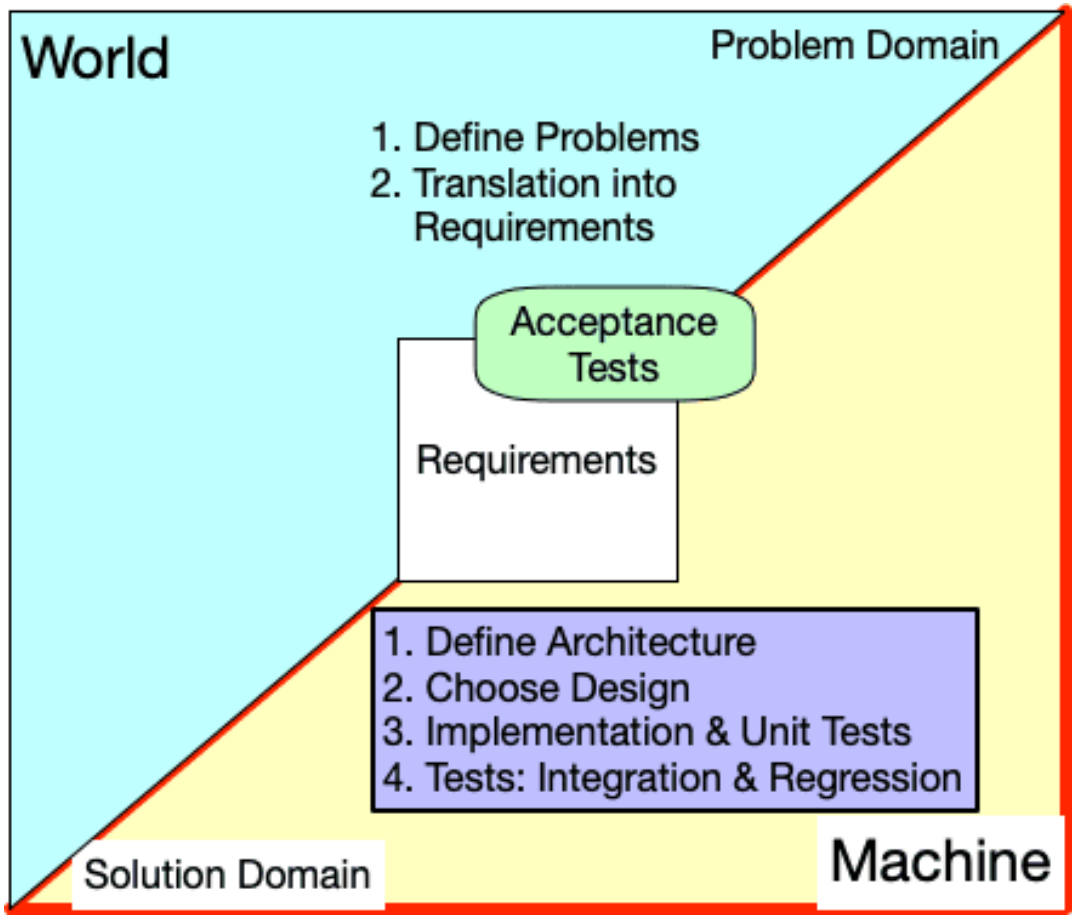
Step 2: Requirements → Design

Choosing the Machine (Solution Domain)

Machine = the software system we build to solve the problem.

With requirements defined as user stories:

- Architect and design the Machine (software)
- Implement the design
- Verify implementation against the architecture and design


World
Problem Domain
1. Define Problems
2. Translation into Requirements
Acceptance Tests
Requirements
1. Define Architecture
2. Choose Design
3. Implementation & Unit Tests
4. Tests: Integration & Regression
Solution Domain
Machine

From Requirements to Architecture

Map each requirement to the components it needs:

- Browser access (F2) → a **web client**
- CRUD schedules (F2) → a **server API** + **database**
- Email reminders (F1) → a **scheduler** + **email service**

A good architecture gives every requirement a home.

Select Architecture

3-Tier web architecture plus a reminder job:

- Client → server (HTTP, REST API)
- Server → database (CRUD)
- **Scheduler** → **email service** (due-date checks)

`Web Client → REST API → Database`

`Scheduler → Email Service`

Data Modeling

- Architecture shows information flow.
- `Data modeling` defines the information itself.
- Include every field required by the requirements.

In `TodoApp`, each schedule needs:

1. **Title** — the action *(F2)*
2. **Date** — when it's due *(F2)*
3. **Email** — where to send reminders *(F1-R1a)*
4. **Reminder Days (N)** — days before the deadline to notify *(F1-R1a)*
5. **Completed** — done or not *(F1-R2)*
6. **Notified** — reminder sent or not *(F1-R1b)*

For simplicity, no login: each schedule stores its own email and N.

Our design creates, processes, stores, and deletes this data.

Choose Design

Here, **design** means **modules & interfaces**:

- Build modules with clear interfaces
- Unit-test each module's important behavior
- Show how modules use the data model

Design also uses OOP principles such as SOLID and tools such as UML. ASE 420 covers these in depth.

Front End Design

Front-end design also needs a usable UI.

- `views/` holds the HTML (GUI) components.

```
── views
    ├── detail.ejs
    ├── edit.ejs
    ├── list.ejs
    ├── nav.ejs
    └── write.ejs
```

Utility Functions & Tests

Utility functions are a key part of application design.

- Give reusable utility modules clear interfaces.
- Test important behavior through each module's interface.

```
├── tests
│   └── db.util.js
├── util
│   └── util.js
...
```

Main Modules

Requests flow in one direction:

`View → Route → API/Service → Database`

```
├── index.js
├── api
│   ├── routes.js
│   └── api.js
```

Step 3: Design → Implementation

Implement the design and write tests as you go.

Choosing the Technology Stack

Why and **what** guide the **how**: pick popular, easy-to-learn tools suited to the problem.

Our web stack:

- **JavaScript** (language), **Node.js** (runtime)
- **MongoDB** (database)
- A scheduler and email service for reminders

Unit Tests

Unit tests check each module's important behavior.

- Most language ecosystems provide test libraries or frameworks.

Step 4: Implementation → Validation

Check each requirement's acceptance criteria, supported by integration and unit tests.

Kinds of Tests

Tests differ by **scope** and **purpose**.

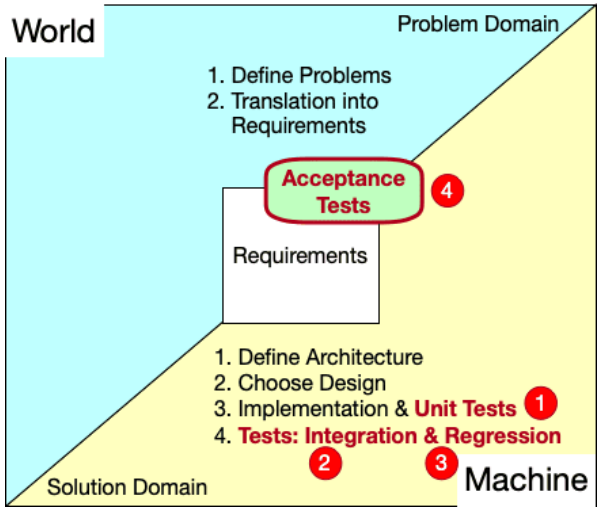
Scope

- Unit: one module
- Integration: several modules
- Acceptance: the whole system, per requirement

Purpose

- Regression: rerun tests after changes (any scope)

- Combine unit, integration, and acceptance tests.
- Rerun them after changes to catch regressions.



Documentation

Good docs preserve project knowledge by answering:

1. What's the goal of this project?
2. How is the project going?
3. What's the architecture & design?
4. Where are the source, tests, and other documents?
5. How do I use this software?

Three Kinds of Documentation

User docs — end users: usage steps, screenshots, troubleshooting

Design docs — developers: architecture, APIs, data model, key decisions

Process docs — team: backlog, sprint notes, progress reports

> **For This Course, You Must Document**

Agile keeps useful documentation without heavy paperwork. Long documents go stale quickly; keep docs useful and current.

Design docs

- Features & requirements (user stories + acceptance criteria)
- Architecture diagram & data model
- Directory structure

User docs

- User manual: prepare for S1P; update for the final

Process docs

- A backlog (done / in progress / planned)
- Short progress notes for your team & instructor

Tests as Design Documents

Well-written tests double as a **living design document**:

- They show expected behavior — input, action, outcome
- They reveal rules and edge cases
- Keep them accurate by updating them when behavior changes

Course Logistics

Related ASE Courses

- ASE 330 (HCI) — UX/UI, so users can use software effectively
- ASE 420 (Software Design) — design rules, tools, and practices
- ASE 456 (Cross-platform Development) — architecture for cross-platform apps

Project Repositories

- GitHub — architecture & design docs, source code, tests
- Canvas Pages — process docs, team progress reports

Portfolio Management

Add project artifacts to your portfolio after each project.

- GitHub Pages offers common, free portfolio hosting.
- Many engineers later use their own domain.